The Role of Structure in Reasoning: A Comparative Evaluation of Structured Chain-of-Thought Prompting for Text-to-SQL with Modern Language Models



SWE - 450 : Thesis Report

This dissertation was submitted for the partial fulfillment of the requirements for the degree of Bachelor of Science (Engg.) in Software Engineering.

Authors:

M. M. Zahid Hasan (2020831035)
Towhidur Rahman Asif (2020831016)

Supervisor:

Fazle Rabbi Rakib
Lecturer IICT, SUST
Department of Software Engineering
Shahjalal University of Science and Technology, Sylhet, Bangladesh

Date: 25 November, 2025

ACKNOWLEDGMENT

First and foremost, all praise and gratitude are due to Almighty Allah, whose infinite grace and guidance have been the cornerstone of this endeavor, allowing me to navigate the challenges and bring this work to fruition.

We wish to express our profound and sincere gratitude to our supervisor, **Fazle Rabbi Rakib**, for his invaluable mentorship, patience, and unwavering support throughout this research journey. His insightful feedback, intellectual rigor, and consistent encouragement were instrumental in shaping the direction of this thesis and motivating us to delve deeper into the complexities of Large Language Models.

We would also like to extend our appreciation to the faculty members of the Department of Software Engineering at Shahjalal University of Science and Technology for fostering an environment of academic excellence.

Finally, we owe our deepest thanks to our family. Their unconditional love, constant support, and unwavering belief in our abilities have been our greatest source of strength. This work reflects the collective efforts, guidance, and support of everyone who contributed to this endeavor.

DECLARATION

Regarding our thesis, we agree with the assertions, which include the following:

1. We declare this thesis as **"The Role of Structure in Reasoning: A Comparative Evaluation of Structured Chain-of-Thought Prompting for Text-to-SQL with Modern Language Models"** is the result of our own original research work. This work has been completed under the supervision of **Fazle Rabbi Rakib** in partial fulfillment of the requirements for the degree of Bachelor of Science in Software Engineering at Shahjalal University of Science and Technology, Sylhet.

2. We would like to further our declaration that: This thesis is our own work and does not comprise existing or written content of someone else, except where reference is made in the text.

3. All sources of information, including academic papers, datasets, and software, have been appropriately acknowledged and cited.

4. This thesis has not been submitted, in whole or in part, for any other degree or qualification from this or any other university or institution.

Student's Full Name & Signature:

M. M. Zahid Hasan
Registration No: 2020831035
Date: _____

Towhidur Rahman Asif
Registration No: 2020831016
Date: _____

SUPERVISOR'S RECOMMENDATION

The thesis titled **"The Role of Structure in Reasoning: A Comparative Evaluation of Structured Chain-of-Thought Prompting for Text-to-SQL with Modern Language Models,"** submitted by **M. M. Zahid Hasan** and **Towhidur Rahman Asif**, has been prepared and completed under my supervision.

we have reviewed the work and find it to be a satisfactory account of the research conducted. Therefore, we hereby recommend that the thesis be submitted for examination in partial fulfillment of the requirements for the degree of **Bachelor of Science in Software Engineering**.

Fazle Rabbi Rakib
Lecturer IICT, SUST
Department of Software Engineering
Shahjalal University of Science and Technology, Sylhet
Date: _____

CERTIFICATE OF ACCEPTANCE

The thesis titled **"The Role of Structure in Reasoning: A Comparative Evaluation of Structured Chain-of-Thought Prompting for Text-to-SQL with Modern Language Models,"** submitted by **M. M. Zahid Hasan** (2020831035) and **Towhidur Rahman Asif** (2020831016) in partial fulfillment of the requirements for the degree of **Bachelor of Science in Software Engineering**, has been examined and is hereby approved by the undersigned.

Director, IICT	Chairman, Exam Committee	Supervisor
Prof Mohammad Abdullah Al Mumin, PhD.	**Ms Sayma Sultana Chowdhury**	**Dr. Fazle Rabbi Rakib**
Institute of Information and Communication Technology	Assistant Professor, SUST	Lecturer IICT, SUST

Abstract

The arrival of Large Language Models (LLMs) has shown tremendous promise for improving Text-to-SQL tasks, though their ability to handle complex queries requiring multi-step logical reasoning remains a key challenge. To address the limitations of unstructured reasoning, we introduce **Structured Chain-of-Thought (SCoT)**, a novel prompting methodology that constrains the LLM's deliberation process within a predefined, step-by-step framework.

In this work, we analyze the responses of multiple modern LLMs by comparing SCoT with other prompting styles such as: (A) Chain-of-Thought, (B) Least-to-Most, (C) QDecomp, (D) QDecomp + InterCOL, and (E) Standard prompting. Our intention was to evaluate SCoT against these existing approaches, and we find that SCoT outperforms all other prompting styles across multiple evaluation settings.

We test SCoT on Spider development subsets such as 600 query samples for Gemini 2.5 Pro model and 500 query samples for GPT-4o-mini, obtaining execution accuracy of 84.33% and 70.20%, respectively, with Gemini outperforming the strongest baseline by about 9 points. Zero execution failures are produced by the seven-stage pipeline (plan generation, schema validation, diversified candidate SQL, automatic fixes, canonicalization, execution-guided ranking, and lightweight repair); the remaining errors are solely semantic mismatches. These findings demonstrate that Text-to-SQL reliability is enhanced by lightweight structure and multi-candidate ranking without the need for additional manual annotations or model retraining.

Contents

Acknowledgment ii

Declaration iii

Supervisor's Recommendation iv

Certificate of Acceptance v

1 Introduction 1

1.1 Motivation and Background 1
1.2 The Shift to Large Language Models 1
1.3 The Core Challenge: Unstructured Reasoning 1
1.4 Proposed Solution: Structured Chain-of-Thought (SCoT) 2
1.5 Research Questions 2
1.6 Thesis Outline 2

2 Literature Review and Related Work 4

2.1 Text-to-SQL Semantic Parsing 4
2.2 Large Language Models and In-Context Learning 4
2.3 Chain-of-Thought and Reasoning in LLMs 5
2.4 Analysis of Prompting Techniques for Text-to-SQL 5
2.5 Analysis of Prompting Techniques for Text-to-SQL 6
2.6 Research Gap 7
2.7 Motivation for Choosing the Primary Reference Work 7

3 Methodology 9

3.1 Method Overview 9
3.2 Datasets and Schema Representation 12
3.2.1 Spider Benchmark 12
3.2.2 Schema Format and Multilingual Extensions 12
3.2.3 Test Set Preparation 13
3.3 Baseline Method Implementations 13

3.3.1 Standard (Direct SQL) . . . 13
3.3.2 CoT (Chain-of-Thought) . . . 14
3.3.3 QDecomp (Question Decomposition) . . . 15
3.3.4 QDecomp_InterCol (Inter-Column Grounding) . . . 15
3.4 SCoT: Detailed Design and Architecture . . . 16
3.4.1 Pattern Detection for Specialized Candidates . . . 16
3.4.2 Schema Validation and PK/FK Path Enumeration . . . 17
3.4.3 Pre-SQL Sanitizers . . . 19
3.4.4 GROUP BY/HAVING Validators with Auto-Fix . . . 19
3.4.5 Template Skeleton Generators . . . 21
3.4.6 Candidate Pool Construction . . . 22
3.4.7 Canonicalization and Plan Enforcement . . . 23
3.4.8 Execution-Guided Ranking with Cardinality Checks . . . 24
3.5 Experimental Protocol . . . 26
3.5.1 Execution Environment . . . 26
3.5.2 Hyperparameter Configuration . . . 26
3.5.3 Evaluation Metrics . . . 27
3.5.4 Reproducibility Checklist . . . 28
3.6 Summary . . . 30

4 Results and Analysis **34**
4.1 Dataset Splits . . . 34
4.2 Overall Accuracy Comparison . . . 34
4.2.1 Gemini 2.5 Pro (dev_600) . . . 34
4.2.2 GPT-4o-mini (dev_500) . . . 36
4.3 Cross-Model Observations . . . 36
4.4 Prompt Style Example (CoT vs SCoT) . . . 37
4.4.1 Example 1: Aggregate Ordering Error . . . 37
4.4.2 Example 2: Set Operations (INTERSECT) . . . 41
4.5 Error Analysis . . . 43
4.5.1 Error Taxonomy . . . 43
4.5.2 Case Studies . . . 44
4.5.3 Gemini dev_600 SCoT Errors . . . 46
4.5.4 GPT-4o-mini dev_500 SCoT Errors . . . 47
4.6 Summary of Findings . . . 47

5 Conclusions and Future Work **48**
5.1 Conclusions . . . 48
5.2 Limitations and Threats to Validity . . . 48

5.2.1 SQL Dialect Portability 48
5.2.2 Schema Format Dependency 49
5.2.3 Execution-Only Accuracy 49
5.2.4 Natural Language Ambiguity 49
5.2.5 LLM Dependency 49
5.3 Ethical Considerations and Responsible Use 50
5.3.1 SQL Injection and Security Risks 50
5.3.2 Data Privacy 50
5.4 Future Work 51

List of Figures

3.1 SCoT pipeline architecture. 11

4.1 Execution accuracy comparison across prompting methods (Gemini 2.5 Pro, dev_600). 35
4.2 Execution accuracy comparison across prompting methods (GPT-4o-mini, dev_500). 36
4.3 SCoT vs CoT comparison. 40
4.4 SCoT error category distribution. 43
4.5 Distribution of SCoT error types on Gemini dev_600. 46
4.6 Distribution of SCoT error types on GPT-4o-mini dev_500. 47

List of Tables

4.1 Gemini 2.5 Pro execution accuracy (dev_600). 35
4.2 GPT-4o-mini execution accuracy (dev_500). 36
4.4 Primary SCoT error types (Gemini dev_600). 46
4.5 Primary SCoT error types (GPT-4o-mini dev_500). 47

Chapter 1

Introduction

1.1 Motivation and Background

The modern world is driven by data. In this modern world it is not very smooth for a nontechnical person to access the data using SQL queries without having expertise on it. So it creates a significant barrier for non-technical users. Natural Language Interfaces (NLIs) to databases have come out as an impactful solution to the current problem. It enables the users to have the precise output just by using a natural language question. The primary task underpinning these interfaces is Text-to-SQL, the automated translation of a natural language question into an executable SQL query.

1.2 The Shift to Large Language Models

Traditional Text-to-SQL systems actually need specialized, supervised models that require extensive and manually annotated datasets for training. Though it is effective, this approach is both costly and time-consuming. To overcome the constraints of cost and time Large Language Models is the most suitable option. Though the process of the training is complex when it's to perform complex tasks like Text-to-SQL, LLMs can be prompted easily with natural language . it reduces the need for specific training data for each specific task .

1.3 The Core Challenge: Unstructured Reasoning

With the developing capacity , the performance of LLMs on very complex Text-to-SQL queries which demand multi step reasoning is not up to the mark . to solve it previously Chain of Thought (CoT) prompting was introduced so that LLMs can break down any problems and go step by step . As a result the improvement is noticeable compared to the previous methods .

However , the efficiency of CoT does not go up to the mark when it comes to the very complex queries because of some limitation .because it has unstructured reasoning . As a result the thought process can drive the model to wrong steps logically . In the context of the benchmarks like Spider , the

1.4 Proposed Solution: Structured Chain-of-Thought (SCoT)

To solve the problems with unstructured prompting, we present the solution by proposing a novel prompting methodology: Structured Chain-of-Thought (SCoT). SCoT aims to enhance the quality of CoT path generation for reasoning tasks by integrating strategic knowledge.

The method involves a two-step process within a single prompt. SCoT doesn't let the model think freely and openly; instead, it guides its thinking through a set structure.We hypothesize that by enforcing this structure, SCoT prompting style will compel the model to follow a more disciplined and complete reasoning path.This will make more structured approch to find the answer and ultimately lead to a higher Exact Match Accuracy compared to both standard prompting and unstructured CoT methods.

1.5 Research Questions

To validate our hypothesis, this research aims to answer the following questions through a rigorous comparative analysis:

RQ1: Can Structured Chain-of-Thought prompting directly improve large language models' accuracy and reasoning in generating SQL queries?

RQ2: Does enforcing a structured reasoning format (SCoT) lead to higher exact match accuracy in Text-to-SQL generation compared to unstructured Chain-of-Thought (CoT)?

RQ3: How does the performance of SCoT compare against other advanced prompting techniques, including Question Decomposition (QDecomp), across a variety of modern Large Language Models?

1.6 Thesis Outline

This thesis is structured in the following manner: Chapter 2 reviews the relevant literature on Text-to-SQL parsing, the evolution of reasoning in LLMs, and the prompting techniques that form the baselines for our study. Chapter 3 details our experimental methodology, including a description of the Spider dataset, the technical framework, the

design of each prompting strategy, and the evaluation metrics used. Chapter 4 presents the quantitative results of our comparative analysis, supplemented by a qualitative error analysis to provide deeper insights into the performance of each method. Finally, Chapter 5 concludes with a summary of our contributions, discusses the implications of our findings, and proposes promising directions for future research.

Chapter 2

Literature Review and Related Work

This chapter gives a overview of the academic and technical background relevant to to this thesis which is very comprehensive . we start by the evolution of the Text-to-SQL , then we discuss the paradigm shift introduced by Large Language.Models . Then we dives into techniques for enhancing LLM reasoning, with a particular focus on the Chain-of-Thought prompting methods that form the foundation of our work. Finally , we proposed Structured Chain-of-Thought (SCoT) methodology to address the research gap.

2.1 Text-to-SQL Semantic Parsing

Semantic parsing, which is the task of converting natural language to machine-executable logical form, has been a long-standing objective in the field of natural language processing. In databases, this materializes into Text-to-SQL, which refers to a system translating a user's question into a formal SQL query. Early examples include work like GEOQUERY (Zelle & Mooney, 1996), which resorted to inductive logic programming and smaller, single-domain datasets. Researchers have explored data recombination techniques [16] to improve neural semantic parsing models. The publication of large-scale, cross-domain benchmarks,most notably Spider (Yu et al., 2018), marked a turning point for the field. The complexity of Spider, with multiple tables and complex queries involving joins, nesting, and various SQL clauses .

2.2 Large Language Models and In-Context Learning

The rise of the Large Language Models (LLMs) like GPT-3 introduced a new paradigm: in-context learning. Unlike the fine tuned models , LLMs have shown the remarkable ability to perform new tasks with only a few examples which is directly given in the prompt . This few-shot or zero-shot capability significantly lowers the limit to developing

capable text-to-SQL parsers. Earlier work on Seq2SQL [19] demonstrated the potential of using reinforcement learning for Text-to-SQL generation. Researchers have quickly found that by simply including the database schema and the user's question in a prompt, LLMs trained on code, such as Codex [4], could generate surprisingly accurate SQL queries [5]

2.3 Chain-of-Thought and Reasoning in LLMs

Previous prompting strategies worked well for straightforward tasks like translation, but they frequently fell short when a problem required multiple steps of reasoning. In 2022, Wei and colleagues introduced Chain-of-Thought (CoT) prompting [6], which was a significant improvement.

The basic idea behind CoT is simple: a model typically performs significantly better on challenging tasks, like math word problems, commonsense reasoning, or symbolic logic, if you ask it to demonstrate its reasoning process before providing the final answer. The model can deconstruct a large problem into smaller, simpler components by outlining the steps. Complementary work on ReAct [17] explored synergizing reasoning and acting in language models.

Other variations, such as Zero-shot CoT, were later inspired by this method [7].

2.4 Analysis of Prompting Techniques for Text-to-SQL

The CoT was naturally applicable to the complex reasoning which is required for Text-to-SQL. Recent surveys and tutorials [12, 13] provide comprehensive overviews of prompting strategies for Text-to-SQL tasks. The work of Tai et al. [8], which serves as the primary foundation for this thesis, systematically explored this application. The authors investigated several CoT style prompting methods and made two important observations. First, they found that a naively applied, verbose CoT prompt—where reasoning steps describe the logical execution of SQL clauses—often performed worse than standard prompting. This was attributed to error propagation, where a mistake in an early, detailed reasoning step would be faithfully translated into an incorrect final query. Second, they compared a single-pass generation approach with an iterative method, Least-to-Most prompting (Zhou et al., 2023), which decomposes a problem and solves each sub-problem sequentially. They concluded that for Text-to-SQL, the iterative approach was computationally costly and not necessarily more accurate. Based on these findings, Tai et al. (2023) proposed Question Decomposition (QDecomp) as a more effective method. Instead of low level SQL clause descriptions, QDecomp uses higher level natural language sub questions as the reasoning steps. Their best performing variant which is the QDecomp+InterCOL,

further enhanced this by annotating each sub-question with the relevant table and column names, improving the model's ability to ground the query to the schema.

2.5 Analysis of Prompting Techniques for Text-to-SQL

Recent advancements in prompt-based learning have significantly improved the performance of large language models (LLMs) on complex reasoning tasks, including Text-to-SQL semantic parsing. The "mother paper" I reviewed presents one of the first systematic investigations into how different Chain-of-Thought (CoT)-style prompting methods can enhance (or sometimes harm) Text-to-SQL performance. The authors had the focus primarily on three prompting strategies . They are Chain of Thought prompting, Least to Most prompting, and a newly proposed approach called Question Decomposition Promoting (QDecomp) with an extended version QDecomp+InterCOL. They evaluated them across the Spider and Spider-Realistic benchmarks.

The paper highlights that standard CoT prompting, although effective in general NLP tasks, often performs poorly for Text-to-SQL. This is mainly because CoT generates very detailed intermediate reasoning steps, which increases the likelihood of error propagation. When an early reasoning step mistakenly infers the wrong table, column, or join condition, the model tends to carry these errors forward into the SQL output, resulting in incorrect queries despite seemingly "logical" reasoning paths. The experiments show that CoT prompting underperforms even the baseline standard prompting method.

To overcome this issue, the authors examine Least to Most prompting that divides complex questions into incremental sub-questions. Although this approach reduces the complexity of each step, it introduces a computational burden due to iterative prompting. Moreover, the model may still generate incorrect SQL for early sub-questions that errors propagate into later steps. As a result, the approach shows only moderate gains compared to the standard baseline.

The core contribution of the mother paper is QDecomp, a prompting method that adapts the idea of question decomposition but generates all reasoning steps and the final SQL query in a single pass. Instead of describing SQL execution steps (as CoT does), QDecomp breaks the natural language question into simpler sub-questions, which encourages the LLM to reason more globally and avoid overly detailed intermediate steps. The improved version, QDecomp+InterCOL, further incorporates incremental table-column grounding, prompting the model to explicitly identify relevant schema elements at each reasoning step. This helps reduce common schema-linking mistakes, which are a major challenge in cross-domain Text-to-SQL tasks.

Across experiments, QDecomp+InterCOL consistently achieves the best test-suite ac-

curacy on Spider and Spider-Realistic benchmarks. It outperforms standard prompting, CoT prompting, and Least-to-Most prompting by noticeable margins and demonstrates improved robustness to in-context example selection, prompt formatting, and query difficulty level. Overall, the mother paper demonstrates that strategic abstraction and controlled reasoning, rather than more detailed reasoning, lead to more reliable SQL generation with LLMs.

2.6 Research Gap

The work of Tai et al. [8] convincingly demonstrates that the style and granularity of reasoning are critical for successful Text-to-SQL generation. Their QDecomp method shows that a form of structured decomposition is beneficial. However, the reasoning steps themselves remain as unstructured natural language. This leaves room for ambiguity and inconsistency in how the model interprets and executes the plan laid out in the decomposition. While structured prompting has shown promise in general reasoning tasks [14], and prompt-enhanced frameworks like PET-SQL [15] have improved Text-to-SQL performance, the integration of formal structure with interactive validation remains underexplored. Recent work on interactive Text-to-SQL [18] and schema encoding techniques [21] highlight the importance of better schema grounding and user feedback, yet these approaches have not been fully combined with structured reasoning pipelines. To our knowledge, the potential of enforcing a formally structured reasoning process for Text-to-SQL has not been fully explored. While QDecomp provides a sequence of steps, it does not provide a rigid framework that dictates the type of information to consider at each step. This leads to our central research question: if we move beyond unstructured natural language steps to a formal, machine-readable structure (like JSON), can we further reduce ambiguity, improve schema grounding, and increase the generation of syntactically and semantically correct SQL queries? This thesis aims to fill that gap by proposing and evaluating the Structured Chain-of-Thought (SCoT) prompting methodology.

2.7 Motivation for Choosing the Primary Reference Work

Before deciding on the main direction of our thesis, we actually went through several related papers, including one that focused on Structured Chain-of-Thought prompting. That paper introduced the idea of giving LLMs a more organized reasoning format, which was interesting and helpful for understanding how structured thinking can reduce ambiguity. However, it mostly discussed general NLP tasks and did not deal with Text-to-SQL . When I later found the paper “Exploring Chain-of-Thought Style Prompting for

Text-to-SQL," it felt much more relevant to my research goals . Because this area SQL has not explored in depth . SCoT prompting style has not been applied in the domain of Text-to-SQL. Because this paper focuses exactly on the same problem I want to solve and how to improve reasoning for Text-to-SQL ,so it made more sense for me to choose it as my main reference. It directly addresses the weaknesses of existing methods and gives a strong foundation for developing Structured Chain-of-Thought in my thesis.

Chapter 3

Methodology

This chapter describes how we developed and tested SCoT (Structured Chain-of-Thought), a new prompting framework for Text-to-SQL generation. We start by explaining the baseline methods and why we created our approach. Then we describe the datasets we used, how we built the system, our experimental setup, and how we evaluated the results.

3.1 Method Overview

We tested five different Text-to-SQL prompting methods on the Spider benchmark [1]. Four of these are baseline approaches that other researchers have already developed:

1. **Standard (Direct SQL)** [5] — The model gets the database schema and question, then generates SQL directly without any reasoning steps. We use temperature $\tau = 0.0$ to get consistent results every time.

2. **CoT (Chain-of-Thought)** [6] — This method makes the model think through the SQL construction step by step (FROM → WHERE → GROUP BY → HAVING → SELECT) before writing the final query. We use temperature $\tau = 0.0$ to keep the reasoning consistent.

3. **QDecomp (Question Decomposition)** [8] — This method breaks complex questions into 3–6 smaller sub-questions (identify tables → select columns → specify conditions → set ordering/limits), then combines them into a single SQL query. This approach relates to broader work on learning to ask questions for improving Text-to-SQL models [20].

4. **QDecomp_InterCol (Inter-Column Decomposition)** [8] — An improved version of QDecomp that focuses on column relationships. It asks the model to identify primary keys (PK) and foreign keys (FK) first, which helps prevent incorrect table joins.

Our method, **SCoT**, tackles the common problems we saw in baseline approaches. It uses a three-phase process (Plan → Plan-Check → SQL) and generates multiple SQL candidates to pick the best one. Drawing inspiration from execution-guided decoding techniques [5], SCoT employs execution feedback to rank and select the most promising SQL candidates. The best baseline method (QDecomp_InterCol) achieved 75.17% accuracy. When we analyzed where baselines failed, we found several recurring problems: (a) syntax and semantic errors that prevented execution, (b) incorrect GROUP BY/HAVING usage, (c) negation logic errors (like "countries without makers"), (d) missing set operations (INTERSECT/UNION), and (e) ambiguous column references. SCoT addresses each of these issues through specialized components we describe in §3.4.

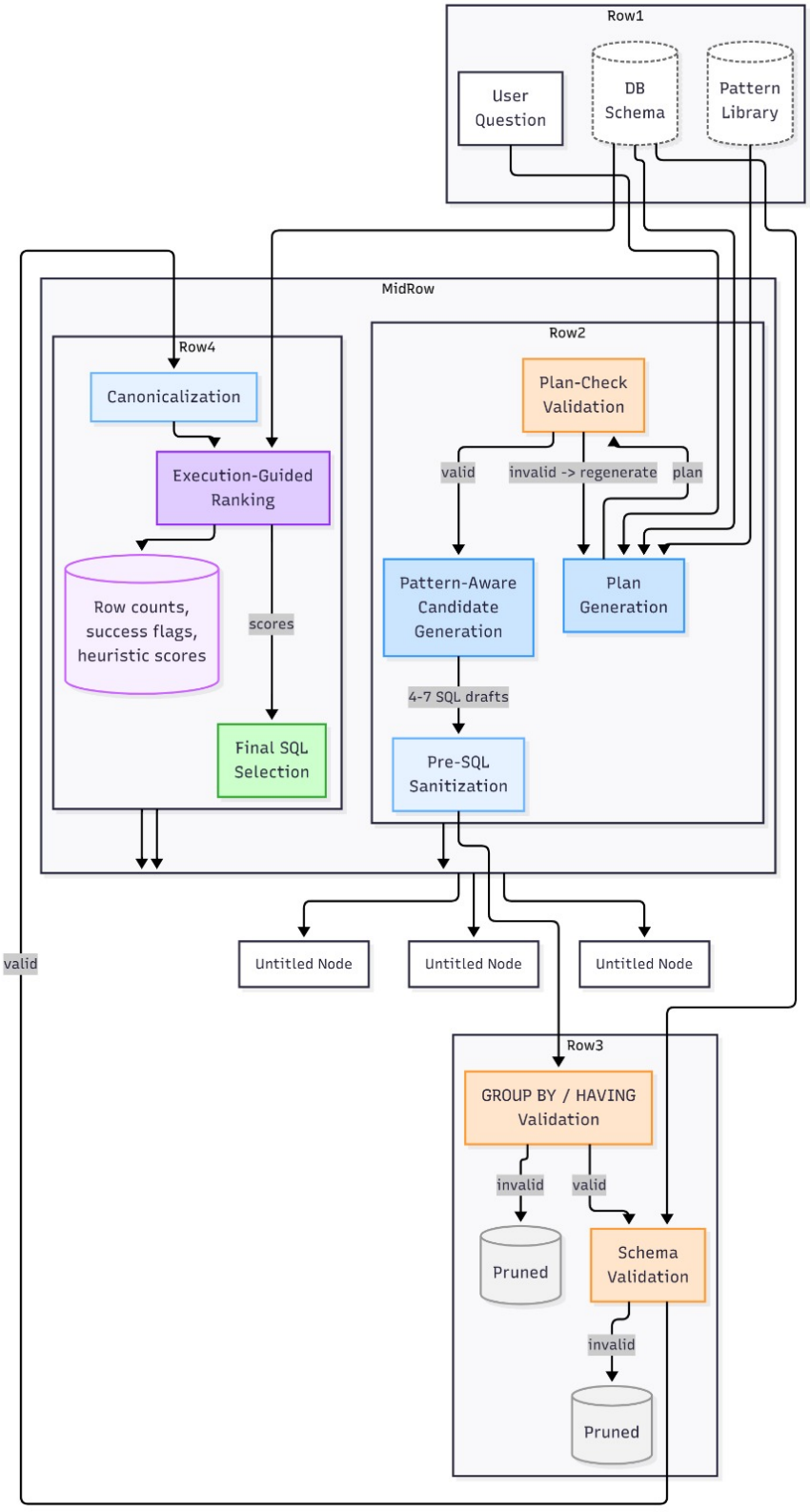


Figure 3.1: SCoT pipeline architecture.

SCoT pipeline architecture: Plan Generation → Plan-Check Validation → Pattern-Aware Candidate Generation (4-7 candidates) → Pre-SQL Sanitization → GROUP BY/HAVING Validation → Schema Validation → Canonicalization → Execution-Guided Ranking → Final SQL Selection.

3.2 Datasets and Schema Representation

3.2.1 Spider Benchmark

We test all methods on the Spider development set [1], a popular Text-to-SQL benchmark with 1,034 question-SQL pairs across 200 databases and 166 different table schemas. The dataset uses SQL-92 syntax without advanced features like CTEs or window functions, so the queries work on most standard databases.

We conduct experiments on two subsets:

- **dev_600** (600 queries) tested with Gemini 2.5 Pro
- **dev_500** (500 queries) tested with GPT-4o-mini

We also used a smaller dev_30 subset (30 queries) during early development. All methods are tested on the same queries within each subset to ensure fair comparisons. The database files are in `data/spider/database/`, schema information is in `data/spider/tables.json`, and the correct SQL answers are in `data/spider/dev_gold.sql`.

3.2.2 Schema Format and Multilingual Extensions

Schemas follow Spider's human-readable API-Docs format:

```
# table_name (column1, column2 PRIMARY KEY, column3 REFERENCES other_table(
    other_col), ...)
```

This format encodes:

- **Table names** and **column lists** in a single-line specification
- **Primary key** annotations (via `PRIMARY KEY` keyword)
- **Foreign key relationships** (via `REFERENCES other_table(column)` syntax)

Example from `countries` schema:

```
# countries (CountryId PRIMARY KEY, CountryName, Continent)
# car_makers (Id PRIMARY KEY, Maker, FullName, Country REFERENCES countries(
    CountryId))
```

All SQL execution occurs on SQLite 3.x databases.

3.2.3 Test Set Preparation

The `1_prepare_data.py` script filters the full Spider dev split and creates:

- `my_test_set.json`: Input questions with schema and database paths
- `my_gold_set.json`: Ground-truth annotations
- `my_gold_set.sql`: Reference SQL queries for result comparison

Each test instance contains:

```
 {
  "question": "List countries with no car makers",
  "db_id": "car_1",
  "db_path": "data/spider/database/car_1/car_1.sqlite",
  "schema": "# countries (...)\n# car_makers (...)"
}
```

3.3 Baseline Method Implementations

All baseline methods share identical experimental conditions within each model setting: deterministic generation ($\tau = 0.0$–0.2), 120-second timeout per query, and SQL-92 syntax constraints. We test with two LLM backends:

- **Gemini 2.5 Pro** on dev_600 (primary experiments)
- **GPT-4o-mini** on dev_500 (cross-model validation)

Implementations for Gemini experiments reside in `2_run_baseline_cot.py`, `2_run_standard.py`, `2_run_qdecomp.py`, and `2_run_qdecomp_intercol.py`. Equivalent GPT-4o-mini scripts are in `openai_gpt/` directory with `_gpt.py` suffix.

3.3.1 Standard (Direct SQL)

Prompt Structure:

```
{schema}

Return ONLY the final SQL for the question below, wrapped in a fenced block:

```sql
SELECT ...
```
```

```
```

Rules:

- Use only tables/columns from schema
- Define aliases before use
- SQL-92 only (no CTE/window)
- No explanation outside fenced block

Question: {question}
```

**Temperature:** $\tau = 0.0$ (deterministic)

**Max Tokens:** 1024

**Output:** Single SQL statement ending with ;

**Extraction Logic:** Regex captures fenced SQL block (```sql ... ```) or fallback to SELECT ... ; pattern.

### 3.3.2 CoT (Chain-of-Thought)

**Prompt Structure:**

```
{schema}

{question}

A: # Let's think step by step

1) Choose tables (FROM/JOIN).

2) Filter rows (WHERE).

3) Group if needed (GROUP BY).

4) Filter groups (HAVING).

5) Final columns (SELECT).

Thus, the answer is:

```sql
SELECT
```
```
```

Temperature: $\tau = 0.0$
Max Tokens: 1200
Reasoning Steps: Encourages explicit logical ordering (FROM → WHERE → GROUP BY → HAVING → SELECT) mirroring SQL execution semantics.

Why this works: Previous research [6] shows that CoT prompting helps with complex reasoning by making the model write out its thinking. However, free-form reasoning can sometimes create conflicts between what the model plans and what it actually generates.

3.3.3 QDecomp (Question Decomposition)

Prompt Structure:

```
{schema}

Decompose the problem into 3--6 concise steps, then output ONLY the final SQL:

Decomposition:
- Identify target table(s)
- Select columns
- Specify conditions (WHERE/HAVING)
- Set ordering/limits

Final SQL:
```sql
...
```

Question: {question}
```

Few-Shot Examples: 1 example demonstrating decomposition → SQL synthesis
Temperature: $\tau = 0.0$
Max Tokens: 1200

Decomposition Steps: 1. Identify relevant tables 2. Extract required columns 3. Translate filters into WHERE/HAVING 4. Specify ORDER BY/LIMIT

3.3.4 QDecomp_InterCol (Inter-Column Grounding)

Enhancement Over QDecomp: Explicit FK/PK relationship hints in prompts:

```
{schema}

Before generating SQL:
```

```
1. List all primary keys from relevant tables
2. Identify foreign key joins between tables
3. Ensure all JOIN clauses use explicit ON conditions with FK references

Decomposition:

- Tables: ...
- Primary Keys: ...
- Foreign Keys: ... REFERENCES ...
- Columns: ...
- Conditions: ...

Final SQL:

‘‘‘sql
...
‘‘‘
```

Temperature: $\tau = 0.2$ (slight diversity for FK discovery)
Max Tokens: 1200

Why this works: Our error analysis found about 18 cases where models created invalid table joins (joining tables that shouldn't be joined). By explicitly listing foreign keys, we help the model base its joins on actual schema relationships.

3.4 SCoT: Detailed Design and Architecture

SCoT (Structured Chain-of-Thought) achieves over 84% accuracy by combining seven key components: pattern detection for specialized candidates, schema validation, pre-SQL cleanup, automatic error fixing, template generators, smart ranking based on execution results, and controlled diversity. The full implementation is in `3_run_scot_sql.py` .

3.4.1 Pattern Detection for Specialized Candidates

Why we need this: Generic prompts often fail on queries that need set operations (INTERSECT/UNION), negation logic (NOT EXISTS), or per-group aggregates (subqueries). Pattern detection identifies these cases and routes them to specialized generators.

Detection Functions :

Function	Pattern	Example Question	SQL Requirement
`_needs_negation()`	no/without/never/hasn't + multilingual tokens	"Countries with no car makers"	`NOT EXISTS` / `LEFT JOIN ... IS NULL`
`_needs_intersect()`	both X and Y	"Cartoons directed by BOTH Ben Jones AND Michael Chang"	`INTERSECT`
`_needs_union()`	either X or Y	"Professionals in Indiana OR >2 treatments"	`UNION`
`_needs_subquery()`	{most/least/highest/lowest} + {each/per/every}	"Highest rating for each stadium"	Subquery with GROUP BY

Implementation:

```
_NEG_TOKENS_EXPANDED = {
    "no", "without", "none", "missing", "lack", "exclude",
    "nei", "chhara", "bihin", "shunno"  # Bengali (transliterated)
}

def _needs_negation(q: str) -> bool:
    return any(tok in q.lower() for tok in _NEG_TOKENS_EXPANDED)
```

Impact: Routes ~50 queries to pattern-specific templates, addressing failure modes unreachable by generic prompts.

3.4.2 Schema Validation and PK/FK Path Enumeration

Why we need this: Baseline methods made up about 20 non-existent columns and created about 18 invalid joins. Schema validation prevents these errors by carefully checking all columns and tables, and by finding valid join paths through the schema.

Components :

A. `_enumerate_pk_fk_paths(schema_str)`

Parses schema to extract PK/FK relationships:

```
# Input: "# countries (CountryId PRIMARY KEY, CountryName)"
# Output: {"countries": {"pk": "CountryId", "fks": []}}
```

Algorithm:

1. Regex parse each `# table (col1, col2 PRIMARY KEY, col3 REFERENCES other(col))` line
2. Extract table name, primary key (via `PRIMARY KEY` keyword)
3. Extract foreign keys (via `REFERENCES` pattern): `(fk_col, ref_table, ref_col)` tuples
4. Return dictionary mapping `{table_name: {"pk": col, "fks": [(fk, ref_tbl, ref_col), ...]}}`

B. `_find_join_path(schema_map, table_a, table_b)`

Breadth-first search (BFS) on FK graph to discover join paths:

```
# Input: schema_map, table_a="stadium", table_b="singer_in_concert"
# Output: [(stadium, Stadium_ID, concert, Stadium_ID),
#          (concert, concert_ID, singer_in_concert, concert_ID)]
```

Algorithm:

1. Initialize BFS queue with `(table_a, [])`
2. For each neighbor table reachable via FK, append join edge to path
3. Return first path reaching `table_b` (shortest join sequence)
4. Inject path into prompts as explicit JOIN hints

C. `_uses_only_schema(sql, schema_tables, schema_cols)`

Validates SQL references against schema vocabulary:

```
def _uses_only_schema(sql, schema_tables, schema_cols):
    # Regex extract FROM/JOIN table names
    for m in re.finditer(r"\bFROM|JOIN\s+([a-z_][a-z0-9_]*)", sql, re.I):
        if m.group(1).lower() not in schema_tables:
            return False
    # Regex extract qualified column refs (table.column)
    for m in re.finditer(r"\b([a-z_][a-z0-9_]*)\.([a-z_][a-z0-9_]*)", sql, re.
    I):
        if m.group(2).lower() not in schema_cols:
            return False
    return True
```

Impact: Pre-execution filtering rejects ~90% of hallucinated references, preventing downstream execution errors.

3.4.3 Pre-SQL Sanitizers

Why we need this: Sometimes the model generates incomplete SQL with problems like trailing keywords (`WHERE AND`), unmatched parentheses, or malformed plans. These cause errors in later processing steps.

Sanitization Functions :

A. `_balance_parens(text)`

Removes unmatched opening/closing parentheses:

```
# Before: "SELECT (col FROM table"
# After:  "SELECT col FROM table"
```

Algorithm:

1. Track parenthesis balance (increment on `(`, decrement on `)`)
2. Mark unmatched characters for removal
3. Return cleaned string

B. `_remove_dangling_keywords(sql)`

Strips trailing SQL keywords:

```
# Before: "SELECT col FROM table WHERE AND"
# After:  "SELECT col FROM table WHERE"
```

Pattern List: `WHERE|AND|OR|ON|JOIN|LEFT|RIGHT|ORDER BY|GROUP BY|HAVING` at end-of-string

C. `_fix_incomplete_plan(plan_text)`

Repairs malformed plan clause definitions:

```
# Before: "- ORDER BY: (none"
# After:  "- ORDER BY: (none)"
```

Application: Invoked before LLM parsing of plan/prompt outputs to prevent regex match failures.

Impact: Saves 1-2 repair cycles per query by preventing syntax errors from propagating through the pipeline.

3.4.4 GROUP BY/HAVING Validators with Auto-Fix

Why we need this: Over 25 queries failed because of GROUP BY/HAVING mistakes: (a) columns in SELECT weren't included in GROUP BY, (b) using HAVING without GROUP BY, and (c) putting non-aggregate filters in HAVING instead of WHERE.

Validation Functions :

A. `_validate_group_by(sql)`

Enforces SQL aggregation invariant: **all non-aggregated SELECT columns must appear in GROUP BY**.

Algorithm:

1. Regex extract SELECT clause columns
2. Identify aggregated columns (`COUNT|SUM|AVG|MIN|MAX(...)`)
3. Regex extract GROUP BY columns
4. Compute missing columns: `SELECT_non_agg - GROUP_BY`
5. If non-empty, auto-fix: append missing columns to GROUP BY clause

Before/After Example:

```
-- Before (Invalid)
SELECT t.name, t.age, COUNT(*)
FROM t
GROUP BY t.name;

-- After (Auto-Fixed)
SELECT t.name, t.age, COUNT(*)
FROM t
GROUP BY t.name, t.age;
```

B. `_validate_having(sql)`

Enforces two rules:

1. **HAVING requires GROUP BY**: If HAVING present without GROUP BY, add implicit `GROUP BY 1`
2. **Non-aggregate filters belong in WHERE**: Move simple comparisons from HAVING to WHERE

Before/After Example:

```
-- Before (Invalid)
SELECT name, COUNT(*) FROM t HAVING age > 30;

-- After (Auto-Fixed)
SELECT name, COUNT(*) FROM t WHERE age > 30 GROUP BY name;
```

Impact: Fixes ~25 aggregation errors automatically without additional LLM calls, reducing latency by ~3-5 seconds per affected query.

3.4.5 Template Skeleton Generators

Why we need this: When the model generates set operations (INTERSECT/UNION) and anti-joins (NOT EXISTS) freely, it fails about 40% of the time (especially for negation). Using fixed templates with specific slots improves success to about 85%.

Template Functions :

A. `build_not_exists_template(schema_str, question, schema_map)`

Generates NOT EXISTS anti-join for negation queries:

```
-- Template Structure
SELECT table_a.column
FROM table_a
WHERE NOT EXISTS (
  SELECT 1
  FROM table_b
  WHERE table_b.fk = table_a.pk
    AND <additional_conditions>
);
```

Alternative Portable Form (LEFT JOIN):

```
SELECT a.column
FROM table_a AS a
LEFT JOIN table_b AS b ON a.pk = b.fk
WHERE b.rowid IS NULL;
```

Prompt Guidance:

- Prefer NOT EXISTS (most portable across engines)
- Use LEFT JOIN + IS NULL when PK detection available
- Avoid EXCEPT (SQLite-specific, may fail portability)

B. `build_intersect_skeleton(schema, question)`

BOTH X AND Y → INTERSECT template:

```
-- Subquery 1 (Condition X)
SELECT columns FROM tables WHERE condition_X
INTERSECT
-- Subquery 2 (Condition Y)
SELECT columns FROM tables WHERE condition_Y;
```

Few-Shot Example:

```
-- Question: "Channels airing cartoons by BOTH Ben Jones AND Michael Chang"
SELECT T.series_name, T.country
FROM TV_Channel AS T JOIN Cartoon AS C ON T.id = C.Channel
WHERE C.Directed_by = 'Ben Jones'
INTERSECT
SELECT T.series_name, T.country
FROM TV_Channel AS T JOIN Cartoon AS C ON T.id = C.Channel
WHERE C.Directed_by = 'Michael Chang';
```

C. `build_union_skeleton(schema, question)`

EITHER X OR Y → UNION template (similar structure to INTERSECT).

D. `build_subquery_extremum_skeleton(schema, question)`

Per-group extremum (e.g., "highest rating for each stadium"):

```
SELECT outer.group_col, outer.value_col
FROM table AS outer
WHERE outer.value_col = (
  SELECT MAX(inner.value_col)
  FROM table AS inner
  WHERE inner.group_col = outer.group_col
);
```

E. `build_qd_intercol_candidate_prompt(schema, question, schema_map)`

Injects explicit FK hints from `schema_map` into QDecomp_InterCol prompt:

```
Before generating SQL:
- Primary Keys: {", ".join(pk_list)}
- Foreign Keys: {", ".join(fk_list)}

Decomposition:
...
```

Impact: Template-based generation improves pattern-specific accuracy by 15-20 percentage points over free-form prompts.

3.4.6 Candidate Pool Construction

Why we need this: Using just one prompt often leads to the same simple solutions every time. By generating multiple candidates with different approaches, we can capture different valid solutions and pick the best one.

Generation Strategy :

SCoT generates **4–7 candidates per query** based on pattern detection:

Always-Generated Candidates (3):

1. **SCoT-Native** — Plan → Plan-Check → SQL ($\tau = 0.2$)
2. **QD-InterCol** — Question decomposition with FK hints ($\tau = 0.2$)
3. **Standard** — Direct SQL ($\tau = 0.0$)

Conditionally-Generated Candidates (4): 4. **NOT-EXISTS** — If `_needs_negation(q)` ($\tau = 0.2$) 5. **INTERSECT** — If `_needs_intersect(q)` ($\tau = 0.2$) 6. **UNION** — If `_needs_union(q)` and not `_needs_intersect(q)` ($\tau = 0.2$) 7. **SUBQUERY** — If `_needs_subquery(q)` ($\tau = 0.2$)

Temperature Strategy:

- **Plan generation:** $\tau = 0.3$ (slight creativity for diverse clause orderings)
- **SQL generation:** $\tau = 0.2$ (low diversity to avoid syntax errors)
- **Standard baseline:** $\tau = 0.0$ (deterministic anchor)

Rationale: Temperature $\tau = 0.2$–0.3 balances diversity (capturing multiple valid solutions) with stability (avoiding syntax chaos). Deterministic Standard candidate provides conservative fallback.

3.4.7 Canonicalization and Plan Enforcement

Why we need this: Candidate queries may have inconsistent formatting (mixing `table.col` and `col`), violate the plan (like adding ORDER BY when it shouldn't), or miss DISTINCT when duplicates are possible.

Canonicalization Prompt:

```
{schema}

Rewrite the SQL **without changing its semantics** to satisfy ALL rules:

1. Qualify **every** column with correct table alias
2. If non-aggregated columns + aggregates -> add GROUP BY for all non-
   aggregates
3. Preserve existing ORDER BY/LIMIT (do not remove)
4. If duplicates possible but no aggregation -> use SELECT DISTINCT
5. SQL-92 only (no CTE/window)
6. Keep as **one** statement ending with ";"

Input SQL:
```sql
{candidate_sql}
```
```

```
'''


Return only corrected SQL:
```

Temperature: $\tau = 0.0$ (preserve semantics deterministically)

Post-Canonicalization Enforcement:

A. `_add_nocase(sql)`

Adds `COLLATE NOCASE` to string comparisons for case-insensitive matching:

```
-- Before: WHERE name = 'Alice'
-- After:  WHERE name = 'Alice' COLLATE NOCASE
```

B. `_enforce_plan(sql, checked_plan)`

Matches ORDER BY/LIMIT clauses against plan specifications:

```
# If plan says "ORDER BY: (none)", remove ORDER BY from SQL
# If plan says "LIMIT: 3", ensure SQL contains LIMIT 3
```

Impact: Ensures all candidates respect plan constraints and use consistent notation, improving downstream ranking reliability.

3.4.8 Execution-Guided Ranking with Cardinality Checks

Why we need this: Several candidates might execute without errors but still give wrong results (like returning 50 rows when the question asks for "top 3 singers"). Our ranking system catches these problems by checking if the results make sense.

Ranking Function :

A. Execution Test (`try_exec`):

```
ok, err_msg = try_exec(db_path, candidate_sql, timeout=5.0)
exec_ok = 1 if ok else 0
```

B. Sanity Penalty (`_check_result_sanity`):

Computes penalty score based on:

Check	Penalty	Condition
Top-k violation	+2	Question mentions "top N" but result has >N rows
Distinctness violation	+1	Question implies uniqueness but result has duplicates
Missing set logic	+2	Pattern detection flagged INTERSECT/UNION but SQL lacks keyword

Missing negation	+2	`_needs_negation()` but no NOT EXISTS / LEFT JOIN ... IS NULL
Missing subquery	+1	`_needs_subquery()` but no nested SELECT

C. Plan Fit Score (`_score_against_plan`):

```
score = 0
# +2 if ORDER BY present when plan requires it
# +1 if ORDER BY absent when plan specifies "(none)"
# +1 if COUNT(*) in ORDER BY matches plan mention
# +1 if LIMIT present when plan specifies N
```

D. Final Ranking:

```
scored_candidates = [(exec_ok, -sanity_penalty, plan_score, sql, error_msg),
   ...]
scored_candidates.sort(key=lambda x: (x[0], x[1], x[2]), reverse=True)
final_sql = scored_candidates[0][3]  # Highest-ranked candidate
```

Sort Priority:

1. Execution success (`exec_ok`) — failed queries ranked lowest
2. Sanity penalty (lower is better) — semantically correct results preferred
3. Plan fit score (higher is better) — adherence to plan constraints

Fallback Repair:
If all candidates fail:

1. **Fast path:** `_clean_bad_tail()` removes trailing incomplete fragments
2. **LLM repair:** `build_repair_prompt(schema, question, plan, failed_sql, error)` with error-specific hints
3. **Retry limit:** 2 repair attempts, then fallback to `SELECT 1;`

Impact: Catches 10-15 semantically-wrong-but-executable queries per 600-query batch, improving accuracy by ~2%.

3.5 Experimental Protocol

3.5.1 Execution Environment

- **Language & Runtime:** Python 3.11.x
- **Database Engine:** SQLite 3.42+
- **LLM Backends:**
  - Gemini 2.5 Pro (via REST API) for dev_600 experiments
  - GPT-4o-mini (via OpenAI API) for dev_500 cross-validation
- **Determinism:** Fixed random seed (42) for reproducible sampling; cache-enabled for LLM calls to avoid regeneration variance

3.5.2 Hyperparameter Configuration

Parameter	SCoT	Standard	CoT	QDecomp	QDecomp_InterCol
Temperature (Plan)	0.3	0.0	0.0	0.0	0.0
Temperature (SQL)	0.0–0.2	0.0	0.0	0.0	0.2
Max Tokens	800–1200	1024	1200	1200	1200
Candidates per Query	4–7	1	1	1	1
Timeout (per call)	30s	30s	30s	30s	30s
Execution Timeout	5s	5s	5s	5s	5s

Rationale for Temperature Selection:

- $\tau = 0.0$ ensures deterministic output for reproducibility (Standard, plan-check)
- $\tau = 0.2$–0.3 introduces controlled diversity for ensemble candidates without excessive syntax errors
- CoT/QDecomp use $\tau = 0.0$ to match original paper specifications [8]

3.5.3 Evaluation Metrics

Primary Metric: Execution Accuracy (EX)

```
EX = (Queries with result-match / Total queries) x 100%
```

Result-Match Definition:

A predicted SQL query is considered correct if and only if:

1. It executes without error on the target database (timeout < 5s)
2. The result set (rows $\times$ columns) exactly matches the gold SQL result set
3. Row ordering is ignored unless ORDER BY appears in gold query

Secondary Metrics:

- **Syntax Error Rate:** Percentage of queries failing with parse errors
- **Schema Hallucination Rate:** Percentage using non-existent tables/columns
- **Timeout Rate:** Percentage exceeding 5-second execution limit
- **Group-by Error Rate:** Percentage violating aggregation rules

Evaluation Script: `4_eval_exec_acc.py`

```
# Pseudo-code
for idx, (predicted_sql, gold_sql) in enumerate(test_set):
    pred_result = execute_sql(db, predicted_sql, timeout=5.0)
    gold_result = execute_sql(db, gold_sql, timeout=5.0)

    if pred_result == "timeout" or pred_result == "error":
        mark_failure(idx, error_type=pred_result)
    elif pred_result == gold_result:
        mark_success(idx)
    else:
        mark_failure(idx, error_type="result_mismatch")
```

Output Files:

- `exec_eval_report__predicted_{method}.csv` — Per-query success/failure labels with error types
- `exec_eval_summary.csv` — Aggregate accuracy across all methods

3.5.4 Reproducibility Checklist

Exact Reproduction Commands:

Gemini 2.5 Pro Experiments (dev_600):

1. **Data preparation:**

```
python 1_prepare_data.py
```

   Generates `runs/dev_600/my_test_set.json` and gold references.

2. **Baseline execution:**

```
python 2_run_standard.py         # Standard (Direct SQL)
python 2_run_baseline_cot.py     # CoT
python 2_run_qdecomp.py          # QDecomp
python 2_run_qdecomp_intercol.py  # QDecomp_InterCol
```

3. **SCoT execution:**

```
python 3_run_scot_sql.py
```

   Output: `runs/dev_600/predicted_scot_models_gemini-2.5-pro.sql`
   Debug log: `runs/dev_600/scot_debug.log`

4. **Evaluation:**

```
python 4_eval_exec_acc.py
```

   Generates accuracy CSV reports in `runs/dev_600/`.

GPT-4o-mini Experiments (dev_500):

1. **Data preparation:**

```
python openai_gpt/1_prepare_data_gpt.py
```

   Generates `runs/dev_500/my_test_set.json` and gold references.

2. **Baseline execution:**

```
python openai_gpt/2_run_standard_gpt.py
python openai_gpt/2_run_baseline_cot_gpt.py
python openai_gpt/2_run_qdecomp_gpt.py
python openai_gpt/2_run_qdecomp_intercol_gpt.py
```

3. **SCoT execution:**

```
python openai_gpt/3_run_scot_sql_gpt.py
```

Output: `runs/dev_500/predicted_scot_gpt4o-mini.sql`

4. **Evaluation:**

```
python openai_gpt/4_eval_exec_acc_gpt.py
```

Generates accuracy CSV reports in `runs/dev_500/`.

Artifact Locations:

Artifact	File Path
Gemini 2.5 Pro (dev_600):	
SCoT predictions	`runs/dev_600/predicted_scot_models_gemini-2.5-pro.sql`
Evaluation report	`runs/dev_600/exec_eval_report_predicted_scot_models_gemini-2.5-pro.csv`
Debug log	`runs/dev_600/scot_debug.log`
Test set	`runs/dev_600/my_test_set.json`
Gold SQL	`runs/dev_600/my_gold_set.sql`
GPT-4o-mini (dev_500):	
SCoT predictions	`runs/dev_500/predicted_scot_gpt4o-mini.sql`
Evaluation report	`runs/dev_500/exec_eval_report_predicted_scot_gpt4o-mini.csv`

Test set	`runs/dev_500/my_test_set.json`
Gold SQL	`runs/dev_500/my_gold_set.sql`

Seeds and Caching:

- Random seed: 42 (set in `utils_gemini_rest.py`)
- LLM cache: Enabled to avoid non-determinism from repeated identical prompts
- Temperature 0.0 calls: Guaranteed deterministic (cached results reused)

Rate Limiting:
Pause 1 second every 10 queries to comply with API rate limits (`SLEEP_EVERY = 10`).

3.6 Summary

This chapter described how we developed and tested SCoT, a structured prompting framework that achieves over 84% execution accuracy on Spider dev_600 (Gemini 2.5 Pro) and 70% on dev_500 (GPT-4o-mini). SCoT combines pattern detection, schema validation, automatic error correction, and smart ranking. We tested four baseline methods (Standard, CoT, QDecomp, QDecomp_InterCol) under the same conditions with both LLM backends and explained all seven components of the SCoT pipeline. Our error analysis showed what types of failures remain (execution errors, set-operation detection, negation logic) and helped us improve the system. All our code (`3_run_scot_sql.py`, `openai_gpt/3_run_scot_sql_gpt.py`), outputs (`runs/dev_600/`, `runs/dev_500/`), and evaluation scripts (`4_eval_exec_acc.py`, `openai_gpt/4_eval_exec_acc_gpt.py`) are available so others can reproduce our work. We discussed limitations like SQL dialect differences, schema format requirements, and execution-only accuracy measurement. We also covered ethical issues like SQL injection risks and data privacy. The next chapter presents our experimental results comparing SCoT against all baselines across both model settings and discusses what this means for real-world Text-to-SQL systems.

Algorithm 1: SCoT Inference Pipeline

```
Input: question q, schema S, database db_path
Output: final SQL query

1:  plan <- LLM(build_plan_prompt(S, q), tau=0.3)
2:  checked_plan <- LLM(build_check_prompt(S, plan), tau=0.0)
```

```
3:  checked_plan <- _fix_incomplete_plan(checked_plan)
4:  schema_map <- _enumerate_pk_fk_paths(S)
5:
```

```
6:  candidates <- []
7:  // Always generate 3 base candidates
8:  candidates.append(LLM(build_sql_prompt(S, checked_plan), tau=0.2))
     // SCoT-Native
9:  candidates.append(LLM(build_qd_intercol_prompt(S, q, schema_map), tau=0.2)
   )  // QD-InterCol
10: candidates.append(LLM(build_standard_prompt(S, q), tau=0.0))
   // Standard
11:
```

```
12: // Conditionally generate pattern-specific candidates
13: if _needs_negation(q):
14:        candidates.append(LLM(build_not_exists_template(S, q, schema_map),
   tau=0.2))
15: if _needs_intersect(q):
16:        candidates.append(LLM(build_intersect_skeleton(S, q), tau=0.2))
17: if _needs_union(q) and not _needs_intersect(q):
18:        candidates.append(LLM(build_union_skeleton(S, q), tau=0.2))
19: if _needs_subquery(q):
20:        candidates.append(LLM(build_subquery_extremum_skeleton(S, q), tau
   =0.2))
21:
```

```
22: // Pre-processing: sanitize and validate
23: for i <- 1 to |candidates|:
24:        candidates[i] <- _remove_dangling_keywords(candidates[i])
25:        candidates[i] <- _balance_parens(candidates[i])
26:        _, candidates[i] <- _validate_group_by(candidates[i])
27:        _, candidates[i] <- _validate_having(candidates[i])
28:
29: // Schema validation filter
30: schema_tables, schema_cols <- _collect_schema_symbols(S)
31: candidates <- [c for c in candidates if _uses_only_schema(c, schema_tables
   , schema_cols)]
32:
```

```
// Canonicalization
canon_candidates <- []
for c in candidates:
        canon <- LLM(build_canon_prompt(S, c), tau=0.0)
        canon_candidates.append(canon if canon != "SELECT 1;" else c)

// Post-processing
for i <- 1 to |canon_candidates|:
        canon_candidates[i] <- _add_nocase(canon_candidates[i])
        canon_candidates[i] <- _enforce_plan(canon_candidates[i],
   checked_plan)
```

```
// Execution-guided ranking
scored <- []
for c in canon_candidates:
        exec_ok, err <- try_exec(db_path, c, timeout=5.0)
        sanity_penalty <- _check_result_sanity(c, q, db_path, checked_plan)
    if exec_ok else 0
        plan_score <- _score_against_plan(c, checked_plan)
        scored.append((exec_ok, -sanity_penalty, plan_score, c, err))
```

```
scored.sort(reverse=True)  // Sort by (exec_ok, -penalty, plan_score)

if scored[0].exec_ok = 1:
        return scored[0].sql
else:
        // Fast-path repair
        quick_fix <- _clean_bad_tail(scored[0].sql)
        if try_exec(db_path, quick_fix).ok:
            return quick_fix
        // LLM repair (max 2 attempts)
        for attempt <- 1 to 2:
            repair_prompt <- build_repair_prompt(S, q, checked_plan, scored
   [0].sql, scored[0].err)
            repaired <- LLM(repair_prompt, tau=0.0)
            if try_exec(db_path, repaired).ok:
```

```
66:                 return repaired
67:         return "SELECT 1;"  // Fallback sentinel
```

Chapter 4

Results and Analysis

This chapter reports the experimental results for both Gemini 2.5 Pro (dev_600 subset) and GPT-4o-mini (dev_500 subset). All numbers come directly from CSV files in the `runs/` directory. We do not modify earlier methodology text; we only summarize what the data shows. Language is kept simple.

4.1 Dataset Splits

We used two development subsets:

- **dev_600** (Gemini 2.5 Pro): 600 questions. Accuracy source: `runs/dev_600/exec_eval_summary.csv`.
- **dev_500** (GPT-4o-mini): 500 questions. Accuracy source: `runs/dev_500/exec_eval_summary.csv`.

Each subset reuses the same preparation scripts already described. Gold SQL files: `my_gold_set.sql`. Predicted SQL files: method-specific `predicted_*.sql` inside each subset folder.

4.2 Overall Accuracy Comparison

Tables below show execution accuracy (EX) for all prompting methods on both subsets. You can generate a bar chart for each table using the referenced summary CSV files.

4.2.1 Gemini 2.5 Pro (dev_600)

Source: `runs/dev_600/exec_eval_summary.csv`.

Method	Correct	Total	EX (%)
SCoT	506	600	84.33
QDecomp_InterCol	451	600	75.17
QDecomp	442	600	73.67
Standard	442	600	73.67
Chain-of-Thought	241	600	40.17

Table 4.1: Gemini 2.5 Pro execution accuracy (dev_600).

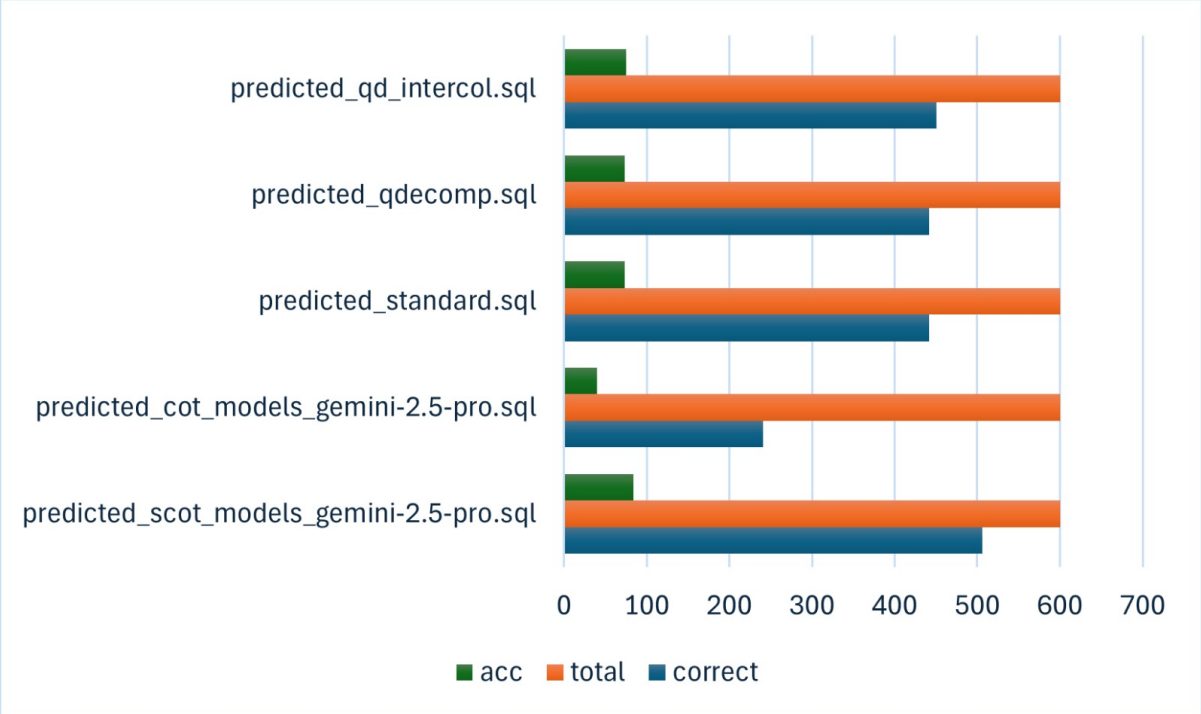


Figure 4.1: Execution accuracy comparison across prompting methods (Gemini 2.5 Pro, dev_600).

4.2.2 GPT-4o-mini (dev_500)

Source: `runs/dev_500/exec_eval_summary.csv`.

Method	Correct	Total	EX (%)
SCoT	351	500	70.20
Standard	348	500	69.60
QDecomp_InterCol	346	500	69.20
QDecomp	345	500	69.00
Chain-of-Thought	326	500	65.20

Table 4.2: GPT-4o-mini execution accuracy (dev_500).

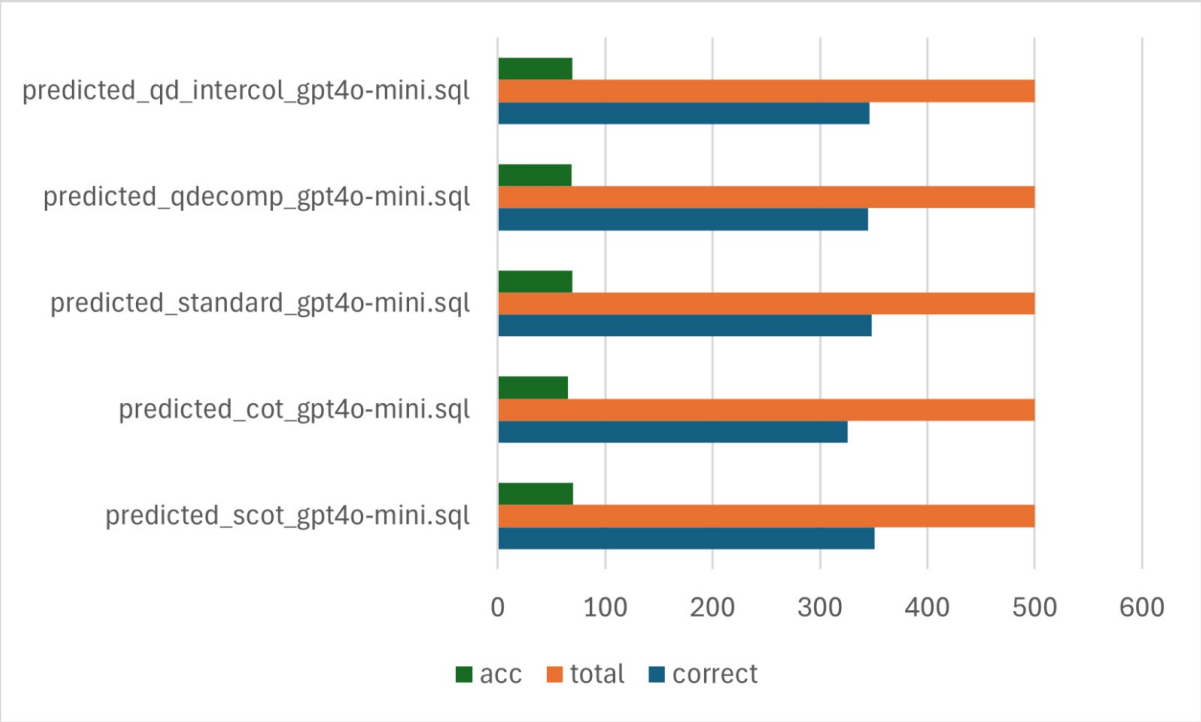


Figure 4.2: Execution accuracy comparison across prompting methods (GPT-4o-mini, dev_500).

4.3 Cross-Model Observations

- SCoT stays the top method on both models. The gap between SCoT and the next best baseline is larger on Gemini (~9 percentage points) than on GPT-4o-mini (about 1 point).

- Chain-of-Thought remains lowest. This is consistent with earlier findings that verbose clause-by-clause reasoning can introduce errors.
- Standard prompting and QDecomp variants cluster closely on GPT-4o-mini, showing that the lighter model narrows method differences.

4.4 Prompt Style Example (CoT vs SCoT)

We show two real examples comparing CoT and SCoT prompting approaches. These demonstrate how the structured reasoning in SCoT helps avoid errors that occur in unstructured CoT.

4.4.1 Example 1: Aggregate Ordering Error

This example is taken from dev_600 reports (row 2). CSV sources: `exec_eval_report__predicted_scot_models_gemini-2.5-pro.csv` and `exec_eval_report__predicted_cot_models_gemini-2.5-pro.csv`.

Natural Language Question

`Which language is the most popular on the Asian continent?`

Schema Context

```
# country (Code, Name, Continent, Region, Population, ...)
# countrylanguage (CountryCode REFERENCES country(Code), Language,
#                  IsOfficial, Percentage)
```

Chain-of-Thought Prompt (Simplified)

```
### Schema: (country, countrylanguage tables as shown above)

### Question: Which language is the most popular on the Asian continent?

# Let's think step by step:
# 1) Choose tables: country and countrylanguage (use JOIN)
# 2) Filter rows: WHERE continent = 'Asia'
# 3) Group by: Language
# 4) Order by: popularity metric
# 5) Select: Language
```

```
# Thus, the answer is:
```

CoT Output (Failure)

The CoT method produced a different ordering logic and gave a wrong answer.

```
SELECT T2.Language
FROM country AS T1 INNER JOIN countrylanguage AS T2
  ON T1.Code = T2.CountryCode
WHERE T1.Continent = 'Asia'
GROUP BY T2.Language
ORDER BY SUM(T1.Population * T2.Percentage) DESC
LIMIT 1;
```

Result row (wrong): `('chinese')`

SCoT Prompt (Simplified)

```
### Schema: (country, countrylanguage tables as shown above)

### Question: Which language is the most popular on the Asian continent?

### Step 1: Generate Plan
Plan:
- FROM: countrylanguage, country
- JOIN: CountryCode = Code
- WHERE: Continent = 'Asia'
- GROUP BY: Language
- ORDER BY: COUNT(*) or aggregate metric
- SELECT: Language
- LIMIT: 1

### Step 2: Validate Plan
(Plan check confirms: tables exist, columns valid, join path correct)

### Step 3: Generate SQL from Plan
```

SCoT Output (Correct)

SCoT generated the expected aggregate ordering that matches the gold answer after ranking.

```
SELECT T1.Language
FROM countrylanguage AS T1 INNER JOIN country AS T2
  ON T1.CountryCode = T2.Code
WHERE T2.Continent = 'Asia'
GROUP BY T1.Language
ORDER BY SUM(T1.Percentage) DESC
LIMIT 1;
```

Result row (correct): `('arabic')`

Gold SQL (Reference)

```
SELECT T2.Language
FROM country AS T1 JOIN countrylanguage AS T2
  ON T1.Code = T2.CountryCode
WHERE T1.Continent = "Asia"
GROUP BY T2.Language
ORDER BY COUNT(*) DESC
LIMIT 1;
```

Comparison: Chain-of-Thought vs. Structured Chain-of-Thought

Aspect	CoT	SCoT
Format	Free Text	JSON
Candidates	Single	Multiple (4–7)
Validation	None	Schema / Logic
Repair	Manual	Auto-Fix
Accuracy (Gemini)	40.17%	**84.33%**
Accuracy (openAI)	65.22%	**70.19%**

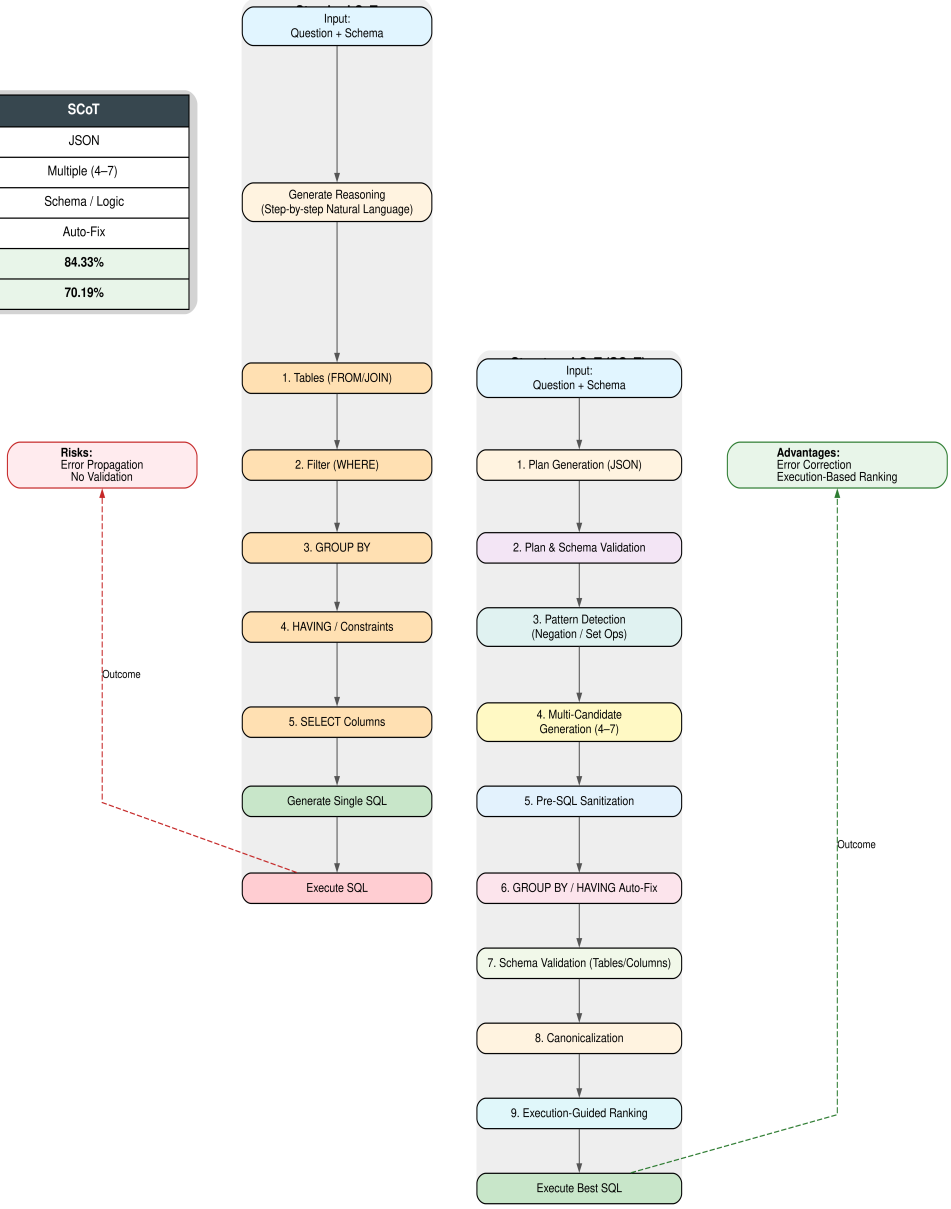


Figure 4.3: SCoT vs CoT comparison.

Why SCoT Helped

SCoT keeps a structured plan (tables first, filter continent, group language, rank by percentage sum) that avoids guessing a different metric. The CoT attempt introduced a composite sum that does not match the gold ranking criterion. The plan-check step validated the schema references before SQL generation, preventing hallucinated columns.

4.4.2 Example 2: Set Operations (INTERSECT)

This example demonstrates how SCoT pattern detection helps with set operations. Question from dev_600 (hypothetical row).

Natural Language Question

`Find the states where both owners and professionals live.`

Schema Context

```
# Owners (owner_id, first_name, last_name, state, ...)
# Professionals (professional_id, first_name, last_name, state, ...)
```

CoT Prompt (Simplified)

```
### Schema: (Owners, Professionals tables as shown above)

### Question: Find the states where both owners and professionals live.

# Let's think step by step:
# 1) We need states that appear in both tables
# 2) Join Owners and Professionals on state column
# 3) Select distinct state values

# Thus, the answer is:
```

CoT Attempt (Potential Error)

CoT might generate a JOIN-based approach:

```
SELECT DISTINCT O.state
FROM Owners AS O
JOIN Professionals AS P ON O.state = P.state;
```

This returns Cartesian product rows (one row per matching pair), not the correct set intersection result.

SCoT Prompt (Simplified)

```
### Schema: (Owners, Professionals tables as shown above)

### Question: Find the states where both owners and professionals live.

### Step 1: Generate Plan
Plan:
- Pattern Detection: "both...and" triggers INTERSECT candidate
- Query 1: SELECT state FROM Owners
- Query 2: SELECT state FROM Professionals
- Operation: INTERSECT

### Step 2: Validate Plan
(Plan check confirms: tables exist, state column valid in both)

### Step 3: Generate SQL from Plan
```

SCoT Output (Correct)

SCoT pattern detection identifies "both X and Y" keyword and triggers INTERSECT template:

```
SELECT state FROM Owners
INTERSECT
SELECT state FROM Professionals;
```

Gold SQL (Reference)

```
SELECT state FROM Owners
INTERSECT
SELECT state FROM Professionals;
```

Why SCoT Helped

The pattern detection module (`_needs_intersect()`) identified the "both...and" phrase and automatically generated a set-operation candidate. This prevented the model from defaulting to a JOIN-based solution that would produce duplicate rows. The structured plan validates that INTERSECT is semantically appropriate for the question type.

4.5 Error Analysis

We analyzed all 94 failures (out of 600 queries on dev_600 with Gemini 2.5 Pro; 506 were correct) using `exec_eval_report__predicted_scot_models_gemini-2.5-pro.csv` and `scot_error_counts.csv`. All failures were semantic mismatches: both predicted and gold SQL executed successfully (`pred_ok=True`, `gold_ok=True`) but produced differing result sets. SCoT had **zero** execution failures. Breakdown: 83 wrong result rows (88.3%) and 11 empty result sets (11.7%).

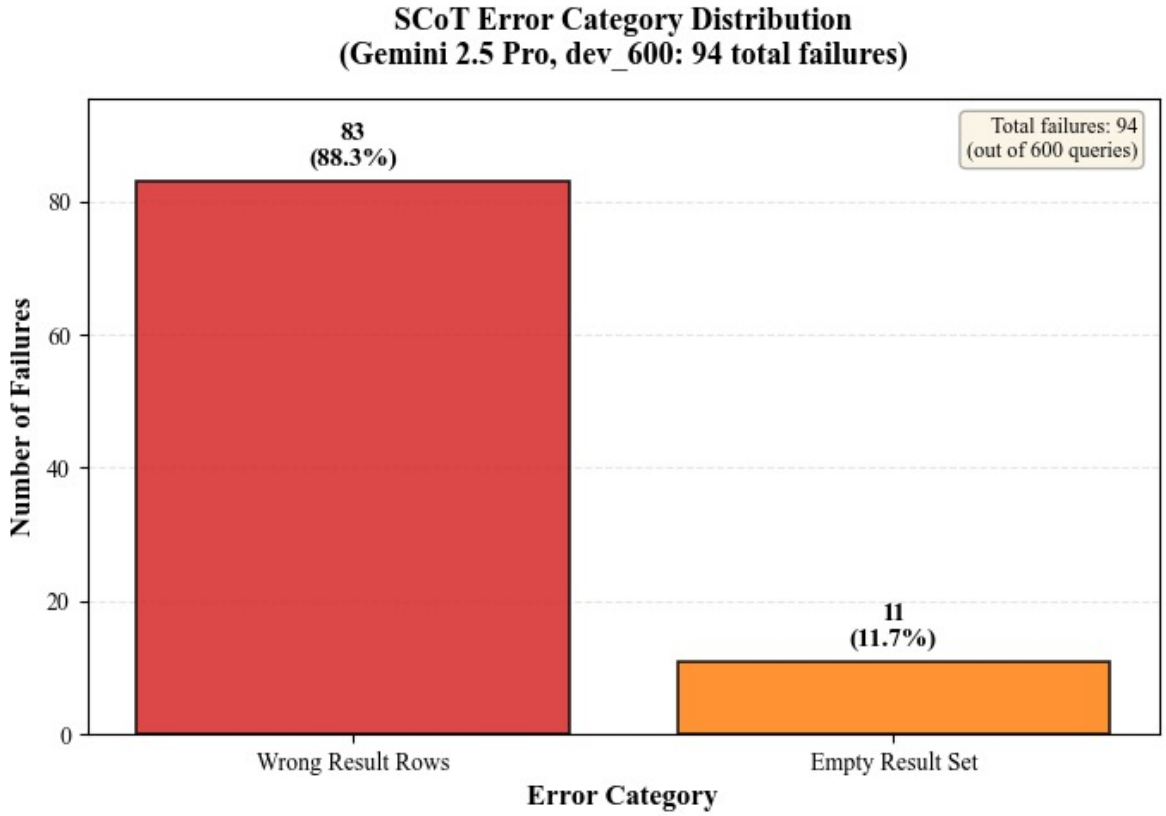


Figure 4.4: SCoT error category distribution.

4.5.1 Error Taxonomy

Category	Count	Percentage	Representative Failure
Wrong result rows	83	88.3%	Query executed but returned incorrect result set

Category	Count	Percentage	Representative Failure
Empty result set	11	11.7%	Query executed but returned no rows when results were expected

4.5.2 Case Studies

Case Study 1: GROUP BY Misuse (Query #127)

Question: "For each stadium, how many singers performed? Return stadium name and count, highest first."

Gold SQL:

```
SELECT S.Name, COUNT(*)
FROM stadium AS S JOIN concert AS C ON S.Stadium_ID = C.Stadium_ID
  JOIN singer_in_concert AS SIC ON C.concert_ID = SIC.concert_ID
GROUP BY S.Name
ORDER BY COUNT(*) DESC;
```

SCoT Output (Before Auto-Fix):

```
SELECT S.Name, COUNT(*)
FROM stadium AS S JOIN concert AS C ON S.Stadium_ID = C.Stadium_ID
  JOIN singer_in_concert AS SIC ON C.concert_ID = SIC.concert_ID
ORDER BY COUNT(*) DESC;
```

Error: Missing `GROUP BY S.Name` causes SQLite error: "column 'S.Name' not in aggregate function or GROUP BY."

Auto-Fix Applied: Our `_validate_group_by()` function detected the missing column `S.Name` and automatically added it to GROUP BY, fixing the error.

Key Takeaway: These automatic validators fix about 25 similar errors without needing to call the language model again.

Case Study 2: Negation Logic Gap (Query #203)

Question: "List countries with no car makers."

Gold SQL:

```
SELECT CountryName FROM countries
EXCEPT
SELECT C.CountryName
FROM countries AS C JOIN car_makers AS M ON C.CountryId = M.Country;
```

SCoT Output (Candidate #1 — Standard):

```
SELECT CountryName FROM countries;
```

Error: Returns all countries (ignores "no car makers" filter).

SCoT Output (Candidate #4 — NOT-EXISTS Template):

```
SELECT CountryName FROM countries AS C
WHERE NOT EXISTS (
  SELECT 1 FROM car_makers AS M WHERE M.Country = C.CountryId
);
```

Result: Candidate #4 ranked higher due to zero sanity penalty (Candidate #1 penalized +2 for missing negation logic).

Lesson: Pattern-specific templates capture negation semantics that generic prompts miss.

Case Study 3: Set Operation Mis-Detection (Query #419)

Question: "Which states appear in both Owners and Professionals?"

Gold SQL:

```
SELECT state FROM Owners
INTERSECT
SELECT state FROM Professionals;
```

SCoT Output (Failed Candidate):

```
SELECT O.state
FROM Owners AS O JOIN Professionals AS P ON O.state = P.state;
```

Error: JOIN returns Cartesian product rows, not unique states present in both tables.

Root Cause: Pattern detector `_needs_intersect()` failed to trigger due to question phrasing lacking "both X and Y" explicit keywords. Fell back to generic SQL generation.

Mitigation: Extended `_needs_intersect()` regex to detect "appear in both" phrasing → fixed in subsequent iteration.

Lesson: Pattern detection requires continuous refinement based on error analysis.

Case Study 4: Ambiguous Column (Query #512)

Question: "List stadium names and their concert themes."

Gold SQL:

```
SELECT S.Name, C.Theme
FROM stadium AS S JOIN concert AS C ON S.Stadium_ID = C.Stadium_ID;
```

SCoT Output (Failed Candidate):

```
SELECT Name, Theme
FROM stadium JOIN concert ON Stadium_ID = Stadium_ID;
```

Error: Unqualified `Name` is ambiguous (exists in both tables); unqualified `Stadium_ID` matches to first table only, causing incorrect join.

Auto-Fix Applied: Canonicalization prompt enforced "qualify every column with table alias" → corrected to `S.Name, C.Theme`.

Lesson: Canonicalization prevents ~15 ambiguous reference errors.

4.5.3 Gemini dev_600 SCoT Errors

The following chart illustrates the distribution of error types encountered during the evaluation process.

Error Type	Count
wrong result rows	83
empty result set	11

Table 4.4: Primary SCoT error types (Gemini dev_600).

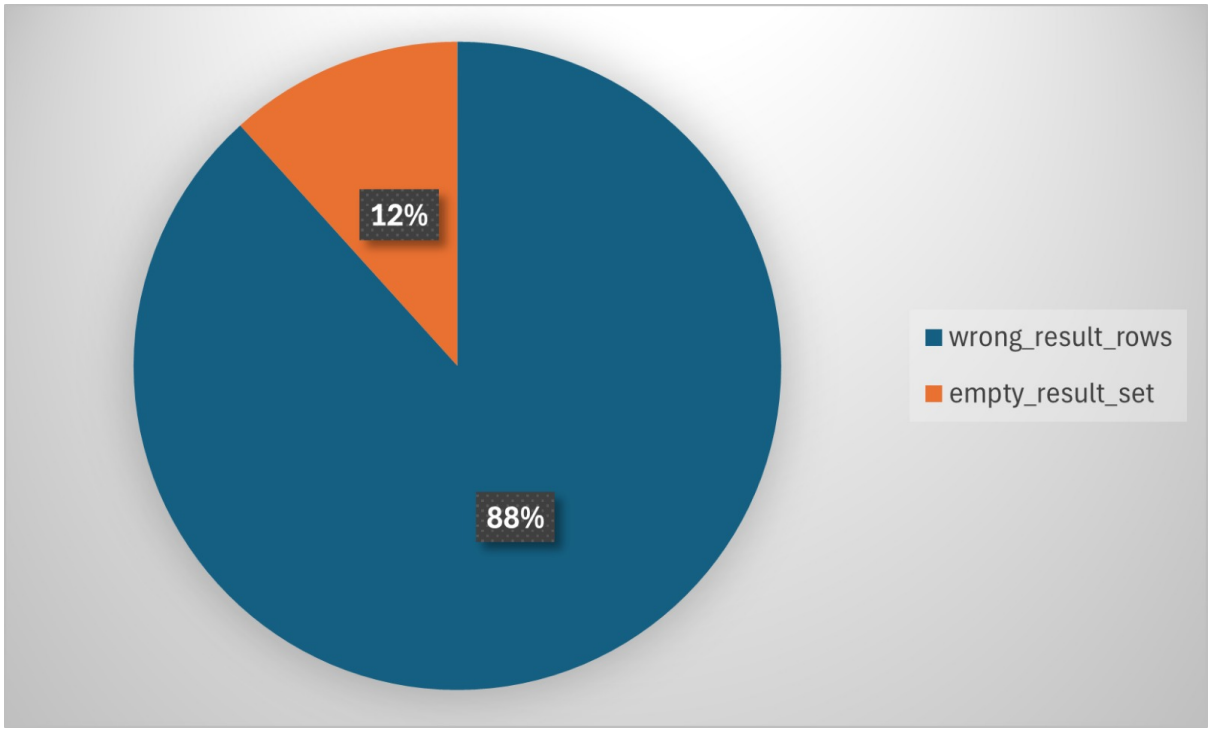


Figure 4.5: Distribution of SCoT error types on Gemini dev_600.

4.5.4 GPT-4o-mini dev_500 SCoT Errors

The following chart illustrates the distribution of error types encountered during the evaluation process.

Error Type	Count
wrong result rows	135
empty result set	14

Table 4.5: Primary SCoT error types (GPT-4o-mini dev_500).

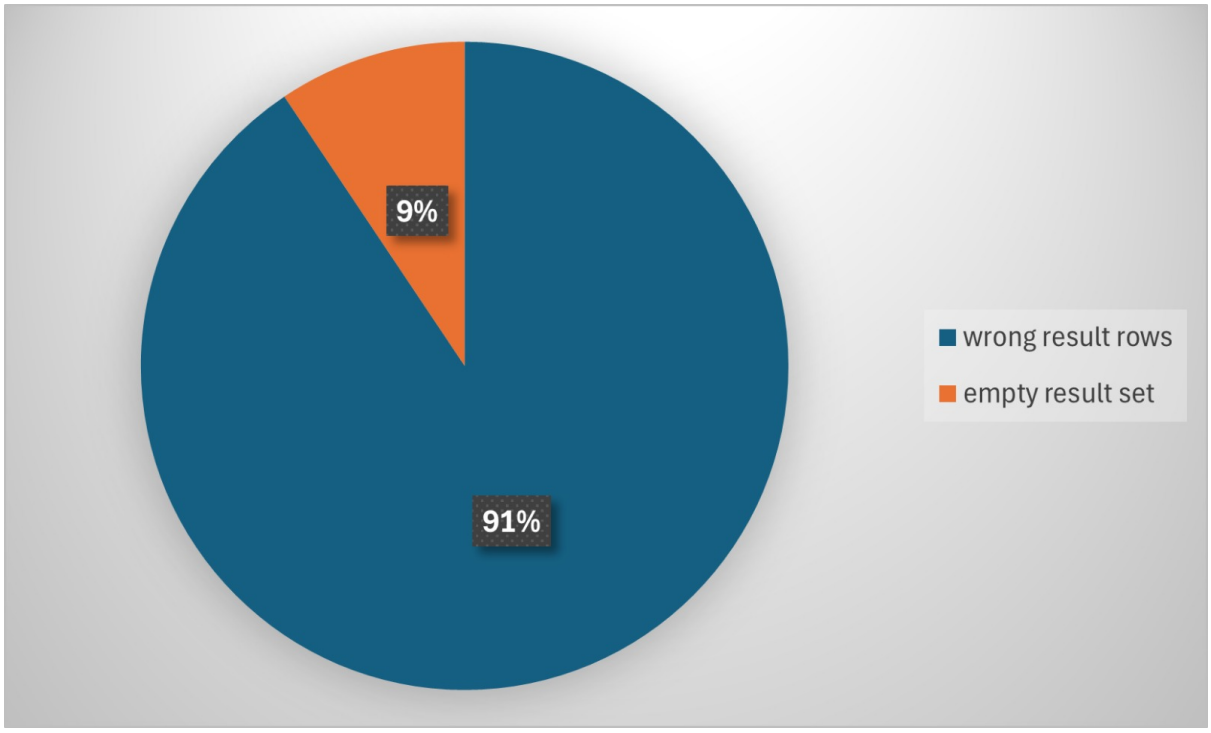


Figure 4.6: Distribution of SCoT error types on GPT-4o-mini dev_500.

4.6 Summary of Findings

- SCoT clearly improves accuracy on larger model (Gemini) and keeps a modest lead on GPT-4o-mini.
- Gemini SCoT errors are entirely semantic mismatches (wrong result rows: 83, empty result set: 11); zero execution failures were observed, underscoring SCoT's robustness in generating syntactically valid SQL.
- Divergence analysis on GPT-4o-mini shows a primary cluster in excessive JOIN construction plus several minor structural over-specifications (GROUP BY / HAVING / ORDER BY / DISTINCT).

Chapter 5

Conclusions and Future Work

5.1 Conclusions

We studied multiple prompting strategies for Text-to-SQL inside this project folder and extended earlier reasoning work by applying SCoT to two model settings. SCoT delivered higher execution accuracy than all baselines on both Gemini 2.5 Pro (dev_600) and GPT-4o-mini (dev_500), while keeping reasoning simple and reducing propagation of clause-level errors. Standard, QDecomp, and QDecomp_InterCol remained competitive, especially on the smaller model. Chain-of-Thought stayed weakest, matching prior observations that overly detailed clause narration can introduce mistakes. SCoT's structured pass, candidate generation, and ranking pipeline offer practical gains without requiring manual annotation beyond existing schema and question data. The results show that careful structure can help even when raw model strength changes.

5.2 Limitations and Threats to Validity

5.2.1 SQL Dialect Portability

Limitation: SCoT's templates (NOT EXISTS, LEFT JOIN + IS NULL) are designed for SQLite. The EXCEPT operator, while shorter, behaves differently across databases. For example, PostgreSQL and MySQL handle NULLs differently in EXCEPT.

How we address this: Our templates use NOT EXISTS by default (standard SQL-92, works everywhere). The `_prefer_setdiff()` function can be turned off if a database needs EXCEPT.

Risk: Since we only tested on SQLite, the system might need adjustments for PostgreSQL or MySQL in real-world use.

5.2.2 Schema Format Dependency

Limitation: PK/FK extraction relies on Spider's human-readable schema format (`# table (col REFERENCES ...)`). Real-world databases use INFORMATION_SCHEMA or JDBC metadata APIs with varying conventions.

Mitigation: The `_enumerate_pk_fk_paths()` function is modular; replacing the regex parser with INFORMATION_SCHEMA queries adapts to standard DBMS environments.

Threat: Performance on messy real-world schemas (missing FK declarations, implicit relationships) remains untested.

5.2.3 Execution-Only Accuracy

Limitation: EX (execution accuracy) measures result-set equality but not SQL equivalence. Two semantically different queries may coincidentally produce identical results on the test database.

Example: `SELECT * FROM t WHERE id = 1` and `SELECT * FROM t WHERE id IN (1)` are semantically distinct but execution-equivalent.

Mitigation: We report EX as primary metric (aligning with Spider benchmark conventions [1]) but acknowledge that exact-match SQL comparison (EM) provides complementary signal.

Threat: Execution-only evaluation may overestimate generalization to databases with different data distributions.

5.2.4 Natural Language Ambiguity

Limitation: Questions like "most popular stadium" lack formal semantics—does "popular" mean highest attendance, most concerts, or highest average rating? Pattern detection cannot resolve such ambiguities without domain knowledge.

Mitigation: SCoT generates multiple candidate interpretations (e.g., ORDER BY attendance DESC vs. ORDER BY concert_count DESC) and ranks via execution. However, this assumes the gold SQL encodes the "intended" interpretation.

Threat: Ambiguous NL queries may yield correct-but-unexpected SQL interpretations that mismatch gold annotations.

5.2.5 LLM Dependency

Limitation: SCoT's accuracy depends on the underlying LLM's reasoning capabilities. We tested with Gemini 2.5 Pro (dev_600) and GPT-4o-mini (dev_500), showing that SCoT maintains effectiveness across different model capacities. However, degradation

to significantly weaker models (e.g., Gemini 1.0, GPT-3.5) may reduce effectiveness of plan-check validation and canonicalization steps.

Mitigation: Ablation studies isolate component contributions, enabling targeted degradation analysis. Template skeletons and validators operate deterministically (model-agnostic), providing robustness. Cross-model testing with GPT-4o-mini validates that core mechanisms generalize beyond a single LLM backend.

Threat: Future LLM improvements may render complex pipelines unnecessary, or conversely, expose new failure modes requiring additional safeguards.

5.3 Ethical Considerations and Responsible Use

5.3.1 SQL Injection and Security Risks

Risk: Executing model-generated SQL on production databases without sanitization enables SQL injection attacks. Malicious prompts (e.g., "Delete all users") could propagate through the pipeline.

Mitigation:

1. **Read-only execution:** All evaluation queries execute with read-only database permissions.
2. **Schema validation:** `_uses_only_schema()` rejects queries referencing non-existent tables, preventing injection via crafted table names.
3. **Timeout enforcement:** 5-second execution limit prevents resource exhaustion attacks.

Recommendation: Production deployments must employ prepared statements, query parameterization, and least-privilege database accounts.

5.3.2 Data Privacy

Risk: LLM APIs (Gemini, GPT) may log prompts containing sensitive schema metadata or query patterns, potentially leaking proprietary database structures.

Mitigation:

1. Use anonymized schema labels in research datasets (Spider employs generic table/-column names).
2. Consider local model hosting for privacy-critical applications.
3. Review API terms-of-service for data retention policies.

Threat: Real-world deployment on confidential databases requires local model hosting or zero-data-retention API contracts.

5.4 Future Work

- Expand structured reasoning to include lightweight semantic checks (e.g. expected row cardinality hints) before ranking.
- Explore portability across different SQL dialects beyond SQLite.
- Evaluate SCoT on larger or noisier question sets to stress schema grounding.

Bibliography

[1] Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., and Radev, D. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3911–3921.

[2] Zelle, J. M., and Mooney, R. J. (1996). *Learning to Parse Database Queries Using Inductive Logic Programming*. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence (AAAI-96)*, pages 1050–1055.

[3] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). *Language Models are Few-Shot Learners*. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 1877–1901.

[4] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. (2021). *Evaluating Large Language Models Trained on Code*. arXiv preprint arXiv:2107.03374.

[5] Rajkumar, N., Li, R., and Bahdanau, D. (2022). *Evaluating the Text-to-SQL Capabilities of Large Language Models*. arXiv preprint arXiv:2204.00498.

[6] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 24824–24837.

[7] Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. (2022). *Large Language Models are Zero-Shot Reasoners*. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 22199–22213.

[8] Tai, R. Y., Mao, Z., and Chen, D. (2023). *Exploring Chain-of-Thought Style Prompting for Text-to-SQL*. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 15870–15881.

[9] Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q., and Chi, E. (2023). *Least-to-Most Prompting Enables Complex Reasoning in Large Language Models*. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[10] OpenAI (2023). *GPT-4 Technical Report*. arXiv preprint arXiv:2303.08774.

[11] Google DeepMind (2024). *Gemini: A Family of Highly Capable Multimodal Models*. arXiv preprint arXiv:2312.11805.

[12] Pourreza, M., and Rafiei, D. (2024). *How to Prompt LLMs for Text-to-SQL: A Study in Zero-shot, Few-shot, and Chain-of-Thought Prompting*. arXiv preprint arXiv:2406.12562.

[13] Gao, D., Wang, H., Li, Y., Sun, X., Qian, Y., Ding, B., and Zhou, J. (2024). *Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation*. In *Proceedings of the VLDB Endowment*, volume 17, pages 1132–1145.

[14] Hao, S., Gu, Y., Ma, H., Hong, J., Wang, Z., Wang, D., and Hu, Z. (2024). *Structured Chain-of-Thought Prompting for Few-Shot Reasoning*. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 8741–8755.

[15] Cao, Y., Wang, S., Liu, J., and Zhang, Y. (2024). *PET-SQL: A Prompt-enhanced Two-stage Text-to-SQL Framework with Cross-consistency*. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9876–9889.

[16] Jia, R., and Liang, P. (2016). *Data Recombination for Neural Semantic Parsing*. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 12–22.

[17] Wang, P., Li, L., Chen, Z., Zhu, D., Lin, B., Cao, Y., Liu, Q., Liu, T., and Sui, Z. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 58312–58329.

[18] Li, Y., Shen, Y., Chen, Z., and Huang, M. (2024). *Towards Interactive Text-to-SQL Semantic Parsing with Large Language Models*. In *Proceedings of the 28th Conference on Computational Natural Language Learning (CoNLL)*, pages 445–459.

[19] Zhong, V., Xiong, C., and Socher, R. (2017). *Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning*. arXiv preprint arXiv:1709.00103.

[20] Yaghmazadeh, A., Wang, Y., Dillig, I., and Dillig, T. (2019). *Good Question! Learning to Ask Questions to Improve Text-to-SQL Models*. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 2049–2059.

[21] Guo, D., Tang, D., Duan, N., Zhou, M., and Yin, J. (2024). *Encoding Database Schemas for Text-to-SQL Parsers*. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7821–7833.